Question 1 [10+10 = 20]

i. Dry run 32-bit assembly code given below, and only reflect changes done to array in given s

```
.data
  array dword -5, 1, 0, 3, 4, -2
  swap db 0
.code

main Proc
 mov ecx, 24
 sub ecx, 4

start:
      mov swap, 0
      mov ebx, 0
loop1:
      mov eax,[ebx+array]
      cmp eax,[ebx+array+4]
      jle noswap

      mov edx,[ebx+array+4]
      mov [ebx+array+4],eax
      mov [ebx+array],edx
      mov swap, 1
noswap:
      add ebx, 4
      cmp ebx, ecx
      jne loop1
      cmp swap, 1
je start
main ENDP
END main
```

(>Here 3 is written but in 4 columns 2 is written. However, the position of either 2 or 3 is same in the final array

Array				
-5	1	0	2	4
-5	0	1	2	4
-5	0	1	2	4
-5	0	1	2	-2
-5	0	1	2	-2
-5	0	1	2	-2
-5	0	1	2	-2
-5	0	1	-2	2
-5	0	1	-2	2
-5	0	1	-2	2
-5	0	1	-2	2
-5	0	-2	1	2
-5	0	-2	1	2
-5	0	-2	1	2
-5	0	-2	1	2
-5	-2	0	1	2
-5	-2	0	1	2
-5	-2	0	1	2
-5	-2	0	1	2
-5	-2	0	1	2
-5	-2	0	1	2
-5	-2	0	1	2
-5	-2	0	1	2

What Program is doing

Performing bubble sort and stops once the array has been sorted & no swaps have been made

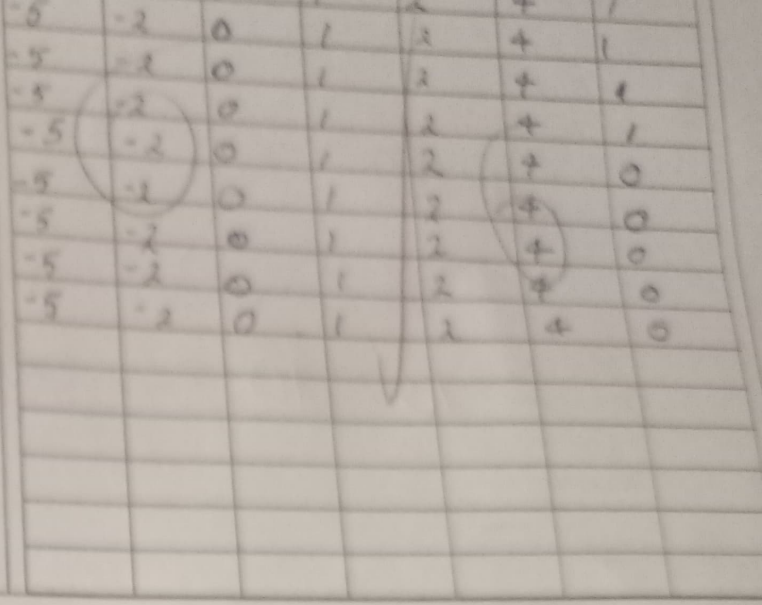
ii. Complete the missing code given below.

a. Complete the **main** that must pass arrays to a common function to proce

```
        cmp ebx, ecx
        jne loop1
        cmp swap, 1
je start
main ENDP
END main
```

What Program is doing

Performing bubble sort and stops once the array has been sorted & no swaps have been made.



Complete the missing code given below.

a. Complete the **main** that must pass arrays to a common function to process passes array. Procedure must receive parameters (array **offset** first then array **length**) through stack.

b. Creating local variable(swap) on the stack in **Function**.

c. After **Ret** from procedure main should retain same old values of Registers as before **CALL** and s should be empty as well.

d. Create local variable on stack for **Swap** (rather than created on memory as shown in **part** i

Note: Filling stack will help you to complete the program also notice line carefully before return

```
.data
    array1 db 8,-6,3,4
    array2 db 9,7,6,5,4,14

.code
main Proc
    mov eax,1
    mov ebx,2
    mov ecx,3
    mov ebp,4
;function calls para passng
    push offset array1
    mov eax, length of array1
    push eax
    call Function

main ENDP
INVOKE ExitProcess,0
```

Address	Content

```
Function proc
;Create local variable
;push register
    push ebp
    mov ebp,esp
    sub esp,4 ; local variable
    push eax
    push ebx
    push ecx

iteration:
 ;accessing local variables
 ;initialization
    mov [ebp-4] ,0
    mov esi , [ebp+12]
  dec ecx mov ecx, [ebp+8]
loop1:
  mov al,[ebx+esi]
  cmp al,[ebx+esi+1]
  jbe noswap

 mov dl,[ebx+esi+1]

 mov [ebx+esi+1],al
 mov [ebx+esi],dl
 mov dword ptr[ebp-4],1

noswap:
 add esi,1
 cmp esi,ecx
 jne loop1

 cmp dword ptr[ebp-4],1
 je iteration

 pop esi
 pop edx
 pop ecx
 pop ebx
 pop eax
 mov esp,ebp
 pop ebp
 ret 8 ; clears st
Function endp
END main
```

Question 2 [10+5 = 15]

Execute complete code and fill/empty the given **STACK**. Update the progress register after executing each line using comma separator. Stack starts at address **0AB0F** All registers initially have **0000H**. Marks will be awarded on detailed execution of the given code.

```
1   .data
2   .code
3   main Proc
4               mov ebx,5
5               mov ecx,4
6               push 2
7               Call Function
8   main ENDP
9   INVOKE ExitProcess,0
10
11  Function proc
12      push ebp
13      mov ebp,esp
14      push ebx
15      push ecx
16      mov eax,[ebp+8]
17      cmp eax,0
18      ja L1
19      mov eax,1
20      jmp L2
21      L1:
22      dec eax
23      push eax
24      inc ecx
25      call Function
26      Return1:
27      mov ebx,[ebp+8]
28      mul ebx
29      L2:
30      pop ecx
31      pop ebx
32      pop ebp
33  ret 4
34  Function ENDP
    END main
```

STACK	
Address	Content
00AB0H	2
00AACH	ret address
00AA8H	0000H
00AA4H	5
00AA0H	4
00A9CH	1
00A98H	ret address
00A94H	00AA8H
00A90H	5
00A8CH	5
00A88H	0
00A84H	ret address
00A80H	0009A6FH
00A7CH	5
00A78H	6

EBP	0000H, 00AA8H, 00A94H, 00A7CH, 0009A4H, 00AA8H
EAX	0000H, 2, 1, 1, 0, 0, 1, 1, 2
EBX	0000H, 5, 5, 1, 5, 2, 5
ECX	0000H, 4, 5, 6, 6, 5, 4

Page 4 of 14

National University of Computer and Emerging Sciences

FAST School of Computing Fall-2023 Islamabad Campus

Question 3 [5+5+5 =15]

Consider the following data declaration, Copy **string1** to **string2** using **STACK** fill given memory after execution of the program.

```
.data
     string1 db "eman srotcurtsni ruoy elcric sunob erocs ot"
     size1=$-string1
     string2 db size1 dup('p')
     last dd 0ABCDH
```

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0100																
0110																
0120																
0130																

Question 4[12 + 3 = 15]

Dry run the given assembly code below? Update memory at every iteration? Write the purpose of the program.

```
.data
      M db 13
      Q db -5
      A db 0
      result dw 0
      Q_1 db 0
.code
main Proc
      mov ax,0
      mov cx,8
      mov al,A
      mov bl,M
      mov dl,Q
      clc
l1:
      mov Q_1,dl
      RCL Q_1,1
      And Q_1,3

      cmp Q_1,1
      jne skipadd

      add al,bl
      jmp shift

skipadd:
      cmp Q_1,2
      jne shift
      sub al,bl
shift:
      SAR al,1
      RCR dl,1
LOOP L1
mov byte PTR result, dl
mov byte PTR result+1,al
```

A	Q	Q_1	M	Steps
0	-5	0	13	0

Purpose of Program

Performing signed division

Question 5[4x3+ 8 = 20]

Consider the given assembly code and update fill memory. For part I, II, III after mov instruction, No marks will be given for mov instruction. You are supposed to understand the given code in Part IV to fill given memory.

Fill Memory and Registers

```
;Part I
.code
 mov ax,0
 mov al,'9'
 add al,'8'
 aaa
 or ax,3030h
```

ax	0107H
ax	3137H

```
;Part II
.data
 val1 db '5'
 val2 db '7'
.code
 mov ax,0
 mov al,val1
 sub al,val2
 aas
 or al,30h
```

ax	FFF9
al	1132H

```
;Part III

.data
 val1 db 5h
 val2 db 6h
.code

 mov ax,0
 mov bl,val1
 mov al,val2
 mul bl
 aam
```

ax	1E0AH
ax	3130H

```
;Part IV
.data
  string1 BYTE "987"
  string2 BYTE "789"
  s3 dB (SIZEOF string1+1)DUP(0),0

.code
main Proc

mov esi,SIZEOF string1 - 1
mov edi,SIZEOF string1
mov ecx,SIZEOF string1
mov bh,0

L1:
   mov ah,0
   mov al, string1[esi]
   add al,bh
   aaa
   mov bh,ah
   add al,string2[esi]
   aaa
   or bh,ah
   or bh,30h
   or al,30h
   mov s3[edi],al
   dec esi
   dec edi
loop L1

mov s3[edi],bh

main ENDP
END main
```

	0	1	2	3	4	5	6	7	8	9	A	B	C	D
0100														
0130														

Question 6 [5+10+5+10+5+10= 45]

i. Suppose AL contains 11001011b and CF=1, write down the new contents of AL after each of the following instructions is executed along with the status of carry flag. Assume the preceding initial conditions for each part of this question:

Instruction	AL (After)	CF
SHL AL,1	1001 0110	1
ROL AL,CL; CL=2	0101 1010	0
ROR AL,CL; CL=3	0100 1011	0
SAR AL,CL; CL=2	0001 0010	1
RCR AL,CL; CL=3	1010 0010	0

ii. Consider following data declaration and fill memory accordingly. ASCII for **A= 41h**.Data memory start at **0100**

00 00 AB CD

CD AB 00

```
.DATA
  v8 label word
  v6 Qword  AB12CD34EF78H,2
  v7 label byte
    v5 DD 0ABCDH, 'ABCD'
    v3 word 0ABH,'AB', 0CDEFH,
              5
    v2 db -1 ,255,-1,12o,16t,0010001b,2
    v1 db 1,2,3,'ABCD',
          0BH,0CH,0DH
       byte "string"
```

```
.CODE
  mov al,sizeof v5
  mov bl,lengthof v5
  mov cl,type v5

  mov al,sizeof v3
  mov bl,lengthof v3
  mov cl,type v3

  mov al,sizeof v1
  mov bl,lengthof v1
  mov cl,type v1

  mov al,v7+1
  mov bx,v8
```

AL	8
BL	2
CL	4
AL	8
BL	4
CL	2
AL	10
BL	10
CL	1

```
mov al, byte ptr v2+3
mov bx, word ptr v5+2
mov ecx, dword ptr v6+1
```

AL	34
BX	AB34
Al	120
BX	00
ECX	12CD 34EF

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4000	78	EF	34	CD	12	AB	00	00	02	00	00	00	00	00	00	00
4010	CD	AB	00	00	CD	AB	00	00	AB	00	AB	00	EF	CD	05	00
4020	-1	255	-1	[illegible]	10	11	2	1	2	3	A	B	C	D	B	C
4030	0	's'	't'	'r'	'i'	'n'	'g'	0	0	0	0	0	0	0	0	0

iii. Update registers and memory after executing the following code. Highlight space in memory created using align.

```
.DATA
count=3
   b1 byte count dup(1)
   align word
   count=1
   b2 db count dup(count dup(1b))
   count=2
   w1 dw count dup('A','B','C')
   count=1
   align word
   d1 DD count dup('ABCD')
```

```
.CODE
   mov eax, offset b2
   mov ebx, offset w1
   mov ecx, offset d1
```

REGISTER

EAX	01000F
EBX	01000B
ECX	011002

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0100	01	01	01	00 (align)	01	0A	00	0B	00	0C	00	0A	00	0B	00	0C
0110	00	00 (align)	41	AB	00	00	00	00	00	00	00	00	00	00	00	00
0120																

iv. Fill in the following table for different caches. Assume all caches are direct mapped. Also rough work is mandatory for each part.

No.	Address Bits	Cache Size	Block Size	Tag Bits	Index Bits	Offset Bits	Bits per Row
a.	16	16KB	8B	2	11	3	67
b.	32	32KB	16B	E17	11	4	146
c.	64	1024 kB	64B	44	14	6	557
d.	32	512KB	32B	23	4	5	280

Rough Work:

a. Offset bits = $\log_2 8$

$B = \frac{16 \text{ kB}}{8} = 2000$

$S = 2000 \Rightarrow \text{index} = \log_2 2000$

b. offset bits = $\log_2 16 = 4$

$B = \frac{32 \times 10^3}{16} = 2000$

$S = 2000 \Rightarrow$ index = 11

v. Convert given C++ Code to assembly equivalent for any value of x and y

```
if(x>=9&&y<15)
    {
        if(x!=7||y>5)
        {
            ax=777;
        }
    }
```

vi. *Write a program that add* **R1=A+B** *and subtract* **R2=C – D,** *and save results. Your computer is 32-bit processor.*

```
.data
A dq 034ABCDEF12H
B dq 078ABCDEF12H
C dq 011123H
D dq 0ABCDEF12H
R1 dq 0
R2 dq 0
.code
```

Question 7 [15]

Show the final data and tags of the two levels of cache, the given memory contents, and data access sequence. Consider 5-bit memory addresses and single byte instructions. Assume that the L1 caches are direct mapped and the L2 cache is fully associative, using LRU for replacement. All caches can place a single byte of data in each block. Also assume that **addresses greater than 19 contain instructions whereas addresses smaller than 19 contain data.**

Memory

Adr.	Data
14	101
15	126
16	3
17	86
18	2
19	91
20	5
21	66
22	7
23	13
24	4
25	42
26	0
27	76
28	12
29	3
30	29
31	17

L2 Cache

Tag	Data
10100	5
01111	126
11110	29
10000	3
11111	17
10001	86
11100	12

L1I Cache

Tag	Data
111	29
1111	17

L1D Cache

Tag	Data
1000	3
0111	126

Access Seq.

Op.	Adr.
FETCH	20
LDRB	15
FETCH	30
LDRB	16
FETCH	31
LDRB	15
FETCH	20
FETCH	17
LDRB	28
FETCH	15
LDRB	30
FETCH	

L1(Instruction) miss rate = 5/6 = 80%

L1(Data) miss rate = 6/6 = 100%

L2 miss rate =

Question 8 [20]

Note: Data is same as address and initial address is 6 Bit for all part of this question.

i. Fill in the **Direct-mapped cache** for the address given below. Calculate Hit and Miss Rate.

Addresses	30H	3FH	32H	38H	39H	33H	3EH	31H	10H	1EH	11H	38H	33H
Address Binary	110000	111111	110010	111000	111001	110011	111110	110001	010000	~~011110~~ 011110	010001	111000	110011
Hit/Miss	Miss	Miss	Miss	Miss	Hit	Hit	Hit	Hit	Miss	Miss	Hit	Hit	Hit

TAG		0	1	V
# of	000	~~30H~~ 10H	~~31H~~ 11H	1
"	001	32H	33H	1
	010			
	011			
"	100	38H	39H	1
	101			
	110			
# of	111	~~3EH~~ 1EH	~~3FH~~ 1FH	1

HIT RATE = Hits / Total Accesses = 7/13 = 0.54 = 54%.

MISS RATE = 6/13 = 0.46 = 46%.

Capacity= 16 Bytes bytes

ii. Fill in the **Direct-mapped cache** for the address given below. Calculate Hit and Miss Rate

Addresses	18H	20H	2FH	1AH	28H	0AH	1BH	18H	0EH	2BH	27H	1FH	2CH
Address Binary	011000	100000	101111	011010	101000	001010	011011	011000	~~010100~~ 001110	101011	100111	011111	101100
Hit/Miss	Miss	Miss	Miss	Hit	Hit	Miss	Hit	Hit	Hit	Miss	Hit	Hit	Hit

TAG	LINE	000	001	010	011	100	101	110	111
1	00	20H	21H	22H	23H	24H	25H	26H	27H
# of 1	01	~~08H~~ ~~28H~~ 28H	~~09H~~ ~~29H~~ 29H	2AH ~~2AH~~ ~~0AH~~	~~0BH~~ ~~2BH~~ 2BH	~~0CH~~ ~~2CH~~ 2CH	~~0DH~~ ~~2DH~~ 2DH	~~0EH~~ ~~2EH~~ 2EH	~~0FH~~ ~~2FH~~ 2FH
	10								
0	11	18H	19H	1AH	1BH	1CH	1DH	1EH	1FH

HIT Rate = 8/13 = 0.62 = 62% Miss Rate= 5/13 = 0.38 = 38%. Capacity= 32 Byte

bits = 1

Fill in the 4 Way Set Associative Cache for the address given below. Calculate Hit and Miss Rate.

Addresses	3FH	24FH	1FH	0FH	3CH	1EH	27H	3DH	05H	3EH	26H	0EH	24H
Address Binary	111 111	100 100	011 111	001111	111100	011 110	100111	111101	000101	111 110	100110	001110	100100
Hit/Miss	Miss	Miss	Miss	Miss	Hit	Miss	Hit	Hit	Miss	Hit	Hit	Miss	Hit

TAG		00	01	10	11	V	LRU
	0						
001	1	0CH	0DH	0EH	0FH	1	01
111	1	3CH	3DH	3EH	3FH	1	10
100	1	24H	25H	26H	27H	1	00
000	1	04H	05H	06H	07H	1	11

HIT RATE = 6/13 = 46% MISS RATE = 7/13 = 54%

iv. Fill in the 8 Way Fully Associative Cache for the address given below. Calculate Hit and MissRate.

Addresses	3FH	24FH	1FH	0FH	3CH	1EH	27H	3DH	05H	3EH	26H	0EH	24H
Address Binary	111 111	100 100	011 111	001 111	111 100	011 110	100 111	111 101	000 101	111 110	100 110	001 110	100 100
Hit/Miss	Miss	Miss	Miss	Miss	Hit	Hit	Hit	Hit	Miss	Hit	Hit	Hit	Hit

TAG	00	01	10	11	V	LRU
111	3CH	3DH	3EH	3FH	1	
1001	24H	25H	26H	27H	1	
011	1CH	1DH	1EH	1FH	1	
0011	0CH	0DH	0EH	0FH	1	
0001	04H	05H	06H	07H	1	000

HIT RATE = 8/13 = 62% MISS RATE = 5/13 = 38%